

this caused problems for coders trying to write graphics software and games. Coding was a nightmare.

Sometime in 1995, Microsoft took note of a small UK-based firm called RenderMorphics that was developing a 3D rendering software called Reality Lab. Once Microsoft was convinced that Reality Lab was just what it needed, it acquired RenderMorphics in February. Reality Lab went on to be renamed Direct3D, and thus began Microsoft's very own 3D API.

To say the beginning was rocky would be an understatement. Since OpenGL was already established by now, was way easier to program for, and was much better and less buggy, it's no surprise that Direct3D got a lot of flak in the beginning. Microsoft's first version of DirectX was numbered 1.0, and contained Direct3D and DirectDraw, for 3D and 2D rendering respectively. However, the release was buggy, slow, and still a lot harder to code for than OpenGL.

It was only with DirectX 7.0 that Microsoft's API was embraced wholeheartedly by game developers. With DirectX 8 and beyond, Microsoft rectified earlier irritations and introduced a set of precompiled code classes that they called Common Files. What these files did was hide all the complicated commands—such as code for listing the capabilities of the hardware. For example, even drawing something as simple as an animated 3D circle can involve hundreds (or even thousands) of lines of coding, even after using older APIs. The Common Files helped by reducing the lines of code required for such common tasks.

The biggest advantage of the DirectX suite of APIs is its seamless integration into all Microsoft operating systems and other platforms such as Xbox. Another advantage is the fact that it's a suite of APIs, so game programmers working with DirectX APIs can code their games for both audio and 3D graphics.

Most importantly, however, is the way DirectX is standardising everything and levelling the playing field for graphics cards manufacturers and game developers alike. Some time ago, you couldn't play most games on lower-end systems, but

now, thanks to DirectX, you are able to play the latest games on any DirectX compliant card—the resolutions and imaging settings will have to be set lower on lower-end cards, but at least you *can* play the games!

Another great innovation brought about by the DirectX family of APIs is the Unified Shader Structure. What this does is let graphics chipsets use the same transistor to do either pixel shading or vertex shading, depending on the API's needs. This definitely gives DirectX (or Direct3D, rather) a definite advantage over OpenGL!

The drawbacks to the DirectX APIs are also the areas in which OpenGL excels.

To begin with, DirectX is not "open" in any way, so any change in the API comes solely at the discretion of Microsoft. Though not necessarily a bad thing, you do have to wait for Microsoft to implement suggested changes, even if a lot of developers are quite capable of doing so on their own.

Although the Windows OS is by far the most popular across the globe, ruling the roost with sheer numbers, not all developers want to limit their games to a single OS. id Software, for example, has always built games that run on just about any OS, and thus have always been in the OpenGL camp. Direct3D also needs a lot more coding than OpenGL from game developers. However, a lot of improvements in the DirectX API as well as the enhanced integration with the operating system mean that more and more games are being made for Windows only. And let's face it: Microsoft's dominance in the OS business doesn't hurt—a lot of game developers today do not consider investing time and money in coding for other OSes a viable option, and are quite happy targeting the masses by coding only for Windows using DirectX/Direct3D.

Platforms

As with most competing technologies, both OpenGL and DirectX have their own fan followings. A lot of open source developers cannot think of using anything but OpenGL, while most game developers seem to be using DirectX as the API of choice. So long as there is a choice of operating

General Purpose Graphics Processors

Graphics Processing Units (GPUs) are very powerful processors, and nothing illustrates this more than the fact that they outdo today's best CPUs from behemoths such as AMD and Intel on a flop-by-flop comparison. In fact, the latest from the nVidia and ATi stables, the 7800 and X1800 respectively, actually severely hammer the fastest CPUs out there—a 7800 GTX achieves about 200 billion floating point operations (flops) a second, while a 3 GHz dual-core Pentium performs just about 20 billion.

It was only a matter of time before someone put 2 and 2 together and arrived at 22! The latest hype of General Purpose GPUs (GPGPUs) is derived from the thought that, "If I have this monster of a GPU, and I'm not playing games all the time, why can't the GPU assist the CPU and make my computer even faster than it already is?"

A valid point! However, it's the architecture of a GPU that lets it down when it comes to general purpose computing. Basically, the example below will illustrate how a GPU is used:

CPU: "Calculate this!"

GPU: "Sir yes sir!"

CPU: "What if I take that number I gave you 1.4 billionths of a second ago and raise it to the power of this new number... what do you think?"

GPU: "Huh?"

Basically, a GPU is optimised for point-to-point operations, and they have very low memory caches, simply because games and 3D rendering just does not require anything more.

So will the GPU ever replace the CPU? Nope! But nothing stops a GPU from helping out now and then!

A CPU can process a couple of threads in parallel, which is contrasted by the latest GPUs, which have 24 pipelines, essentially meaning that they can process 24 threads at a time. If, somehow, we could link the CPU and GPU directly with a PCI-Express communication bridge, and if the CPU and API software could decide whether or not a certain task was better suited for the GPU, you could actually get better performance.

This is the concept of GPGPU implementation, and work is on in full swing using both OpenGL as well as DirectX to try and achieve this. Getting into the nitty-gritty of the changes in the architecture for both CPUs and GPUs as well as the APIs is beyond the scope of this article, but if you're really interested in learning a lot more about this concept, visit www.gpgpu.org for more information!